Plonky2 아키텍처 개요

소개

Plonky2는 Polygon Zero에서 개발한 고성능 SNARK(Succinct Non-interactive ARgument of Knowledge) 증명 시스템입니다. 이 문서는 plonky2의 전체 구조와 핵심 컴포넌트를 설명하여, 영지식증명이 어떻게 구현되고 동작하는지 이해할 수 있도록 합니다.

전체 디렉토리 구조

plonky2 프로젝트는 모노레포(monorepo) 구조로 구성되어 있으며, 여러 개의 독립적인 크레이트(crate)로 나뉘어 있습니다:

```
plonky2/
├── field/ # 유한체(Finite Field) 구현
├── plonky2/ # 핵심 PLONK 증명 시스템
├── starky/ # STARK 증명 시스템
├── maybe_rayon/ # 병렬 처리 추상화
├── util/ # 공통 유틸리티
└── projects/ # 추가 프로젝트 (FFT 최적화 등)
```

1. field 크레이트

유한체 연산의 기본 구현을 제공합니다.

주요 파일:

- `goldilocks_field.rs` : Goldilocks Field (p = 2^64 - 2^32 + 1) 구현
- `goldilocks_extensions.rs` : Goldilocks Field의 확장체 구현
- `fft.rs` : 고속 푸리에 변환(FFT) 구현
- `polynomial/` : 다항식 연산 (보간, 나눗셈 등)
- `arch/x86_64/` : x86-64 아키텍처 최적화

역할:

- 기본 산술 연산 (덧셈, 곱셈, 역원)
- 다항식 연산 및 FFT
- 확장체(Extension Field) 지원
- 하드웨어 최적화 (SIMD, 어셈블리)

2. plonky2 크레이트 (핵심)

PLONK 프로토콜의 핵심 구현이 포함되어 있습니다.

```
plonky2/src/
├── plonk/ # PLONK 프로토콜 핵심
├── fri/ # FRI (Fast Reed-Solomon IOP) 프로토콜
├── iop/ # Interactive Oracle Proof 추상화
├── gates/ # 회로 게이트 구현
├── gadgets/ # 고수준 회로 구성 요소
├── hash/ # 해시 함수 및 Merkle Tree
├── recursion/ # 재귀 증명
└── util/ # 유틸리티 함수
```

3. starky 크레이트

STARK 증명 시스템 구현 (본 문서 시리즈에서는 다루지 않음)

4. maybe_rayon 크레이트

병렬 처리를 위한 추상화 레이어로, Rayon을 사용한 병렬 처리를 선택적으로 활성화할 수 있습니다.

핵심 모듈 상세 설명

plonk 모듈

PLONK 프로토콜의 핵심 로직을 구현합니다.

주요 파일:

1. **circuit_builder.rs**

- `CircuitBuilder<F, D>` 구조체 정의
- 회로 구성을 위한 API 제공
- Target 추가, 연산 정의, 제약조건 설정
- 공개 입력 등록

2. **circuit_data.rs**

- `CircuitData<F, C, D>` : 전체 회로 데이터
- `ProverOnlyCircuitData<F, C, D>` : 증명자 전용 데이터
- `VerifierOnlyCircuitData<C, D>` : 검증자 전용 데이터
- `CommonCircuitData<F, D>` : 공통 데이터
- `CircuitConfig` : 회로 설정 (보안 수준, 와이어 수 등)

3. **prover.rs**

- `prove()` 함수: 증명 생성 메인 로직
- Wire 값 계산
- 다항식 커밋먼트 생성
- Permutation argument (Z 다항식)
- Quotient polynomial 계산

4. [verifier.rs]

- `verify()` 함수: 증명 검증 메인 로직
- 공개 입력 해시 검증
- Vanishing polynomial 평가
- FRI 증명 검증

5. [proof.rs]

- `Proof<F, C, D>` 구조체: 증명 데이터
- `ProofWithPublicInputs<F, C, D>` : 공개 입력을 포함한 증명

6. **permutation_argument.rs**

- Copy constraint를 위한 Permutation argument 구현
- Z 다항식 계산

7. **vanishing_poly.rs**

- Vanishing polynomial 평가
- 모든 제약조건 검증

fri 모듈

FRI (Fast Reed-Solomon Interactive Oracle Proof) 프로토콜 구현

주요 파일:

1. **prover.rs**

- `fri_committed_trees()` : FRI commit phase
- `fri_prover_query_round()` : FRI query phase (prover)
- Folding 과정 구현

2. **verifier.rs**

- `fri_verifier_query_round()` : FRI query phase (verifier)
- `compute_evaluation()` : Evaluation 재계산
- Final polynomial 검증

3. **oracle.rs**

- `PolynomialBatch` : 다항식 배치 관리
- FFT 및 Merkle Tree 구성

4. **proof.rs**

- `FriProof` : FRI 증명 데이터 구조

iop 모듈

Interactive Oracle Proof의 추상화를 제공합니다.

주요 파일:

1. **target.rs**

- `Target` : 회로 내 변수를 나타내는 추상화
- `BoolTarget` : 불리언 타입 Target

2. **witness.rs**

- `PartialWitness<F>` : 부분 Witness (사용자 입력)
- `PartitionWitness<F>` : 전체 Witness (모든 Wire 값)

3. **generator.rs**

- `WitnessGenerator` : Witness 생성 로직
- `SimpleGenerator` : 간단한 Generator trait
- `ConstantGenerator` , `CopyGenerator` 등 기본 Generator

4. **challenger.rs**

- `Challenger<F, H>` : Fiat-Shamir 변환 구현
- 챌린지 생성 및 관리

gates 모듈

회로의 기본 연산 단위인 Gate들을 구현합니다.

주요 Gate 종류:

1. **arithmetic_base.rs**: `ArithmeticGate`

- 기본 산술 연산 (덧셈, 곱셈)
- 가장 많이 사용되는 Gate

2. **arithmetic_extension.rs**: `ArithmeticExtensionGate`

- 확장체에서의 산술 연산

3. **constant.rs**: `ConstantGate`

- 상수 값 정의

4. **public_input.rs**: `PublicInputGate`

- 공개 입력 처리

5. **lookup.rs**: `LookupGate`

- Lookup argument 구현
- Range check 등에 사용

6. **poseidon.rs**: `PoseidonGate`

- Poseidon 해시 함수 구현

gadgets 모듈

고수준의 회로 구성 요소를 제공합니다.

주요 Gadget:

1. **arithmetic.rs**

- 산술 연산 헬퍼 함수

2. **range_check.rs**

- 범위 검증 구현

3. **hash.rs**

- 해시 함수 회로 구성

4. **lookup.rs**

- Lookup table 관리

hash 모듈

해시 함수 및 Merkle Tree 구현

주요 파일:

1. **poseidon_goldilocks.rs**

- Goldilocks Field용 Poseidon 해시

2. **merkle_tree.rs**

- Merkle Tree 구현
- `MerkleCap` : Merkle Tree의 상위 레벨 해시

3. **merkle_proofs.rs**

- Merkle Proof 생성 및 검증

recursion 모듈

재귀 증명 구현

주요 파일:

1. **recursive_verifier.rs**

- 회로 내에서 다른 증명을 검증
- `VerifierCircuitTarget` : 검증자 데이터의 회로 표현

핵심 데이터 구조

1. CircuitBuilder<F, D>

회로를 구성하는 빌더 패턴 구조체입니다.

```
pub struct CircuitBuilder<F: RichField + Extendable<D>, const D: usize> {
pub config: CircuitConfig,
// Gates
gates: HashSet<GateRef<F, D>>,
gate_instances: Vec<GateInstance<F, D>>,
// Targets
virtual_target_index: usize,
// Constraints
copy_constraints: Vec<CopyConstraint>,
// Public inputs
```

```
public_inputs: Vec<Target>,
// Generators
generators: Vec<WitnessGeneratorRef<F, D>>,
// ... 기타 필드
}
```

주요 메서드:

- `new(config: CircuitConfig)` : 새 빌더 생성
- `add_virtual_target()` : 새 Target 추가
- `add()` , `mul()` : 산술 연산
- `connect()` : Copy constraint 추가
- `register_public_input()` : 공개 입력 등록
- `build()` : 회로 빌드

2. CircuitData<F, C, D>

빌드된 회로의 모든 데이터를 포함합니다.

```
pub struct CircuitData<F, C, const D: usize>
where
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
{
pub prover_only: ProverOnlyCircuitData<F, C, D>,
pub verifier_only: VerifierOnlyCircuitData<C, D>,
```

```
pub common: CommonCircuitData<F, D>,
}
```

주요 메서드:

- `prove(inputs: PartialWitness<F>)` : 증명 생성
- `verify(proof: ProofWithPublicInputs<F, C, D>)` : 증명 검증

3. ProverOnlyCircuitData<F, C, D>

증명자만 필요한 데이터입니다.

```
pub struct ProverOnlyCircuitData<F, C, const D: usize>
where
F: RichField + Extendable<D>,
C: GenericConfig<D, F = F>,
{
pub generators: Vec<WitnessGeneratorRef<F, D>>,
pub generator_indices_by_watches: BTreeMap<Target, Vec<usize>>,
// Permutation argument
pub sigmas: Vec<Vec<F>>,
pub subgroup: Vec<F>,
// Commitments
pub constants_sigmas_commitment: PolynomialBatch<F, C, D>,
// FFT
pub fft_root_table: Option<Arc<FftRootTable<F>>>,
```

```
// ... 기타 필드
}
```

4. VerifierOnlyCircuitData<C, D>

검증자만 필요한 데이터입니다 (매우 작음).

```
pub struct VerifierOnlyCircuitData<C, const D: usize>
where
C: GenericConfig<D>,
{
pub constants_sigmas_cap: MerkleCap<C::F, C::Hasher>,
pub circuit_digest: HashOut<C::F>,
}
```

5. CommonCircuitData<F, D>

증명자와 검증자 모두 필요한 공통 데이터입니다.

```
pub struct CommonCircuitData<F: RichField + Extendable<D>, const D: usize> {
pub config: CircuitConfig,
pub fri_params: FriParams,
// Gates
pub gates: Vec<GateRef<F, D>>,
pub selectors_info: SelectorsInfo,
```

```
// Circuit structure
pub degree_bits: usize,
pub quotient_degree_factor: usize,
pub num_gate_constraints: usize,
pub num_constants: usize,
pub num_public_inputs: usize,
// ... 기타 필드
}
```

6. Proof<F, C, D>

생성된 증명 데이터입니다.

```
pub struct Proof<F: RichField + Extendable<D>, C: GenericConfig<D, F = F>, cons
// Merkle caps
pub wires_cap: MerkleCap<F, C::Hasher>,
pub plonk_zs_partial_products_cap: MerkleCap<F, C::Hasher>,
pub quotient_polys_cap: MerkleCap<F, C::Hasher>,
// Openings
pub openings: OpeningSet<F, D>,
// FRI proof
pub opening_proof: FriProof<F, C::Hasher, D>,
}
```

데이터 구조 관계도

```
classDiagram

class CircuitBuilder {

+config: CircuitConfig

+gates: HashSet~GateRef~

+copy_constraints: Vec~CopyConstraint~

+public_inputs: Vec~Target~

+generators: Vec~WitnessGeneratorRef~

+add_virtual_target() Target

+add() Target

+mul() Target

+connect()

+build() CircuitData

}



class CircuitData {

+prover_only: ProverOnlyCircuitData

+verifier_only: VerifierOnlyCircuitData

+common: CommonCircuitData

+prove() ProofWithPublicInputs

+verify() Result

}
```

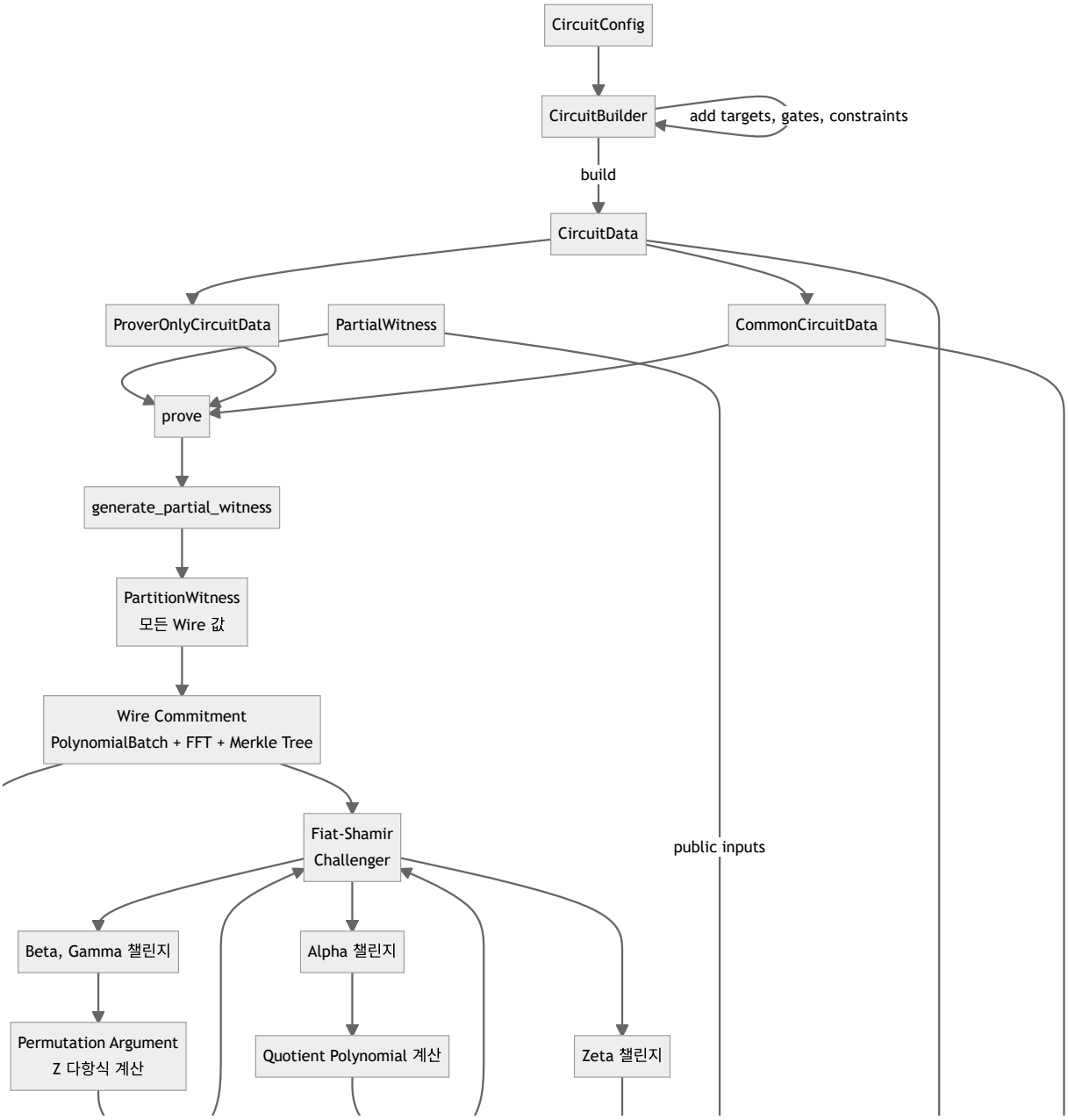
```
class ProverOnlyCircuitData {
+generators: Vec~WitnessGeneratorRef~
+sigmas: Vec~Vec~F~~
+constants_sigmas_commitment: PolynomialBatch
+fft_root_table: FftRootTable
}

class VerifierOnlyCircuitData {
+constants_sigmas_cap: MerkleCap
+circuit_digest: HashOut
}

class CommonCircuitData {
+config: CircuitConfig
+fri_params: FriParams
+gates: Vec~GateRef~
+degree_bits: usize
+quotient_degree_factor: usize
}

class ProofWithPublicInputs {
+proof: Proof
```

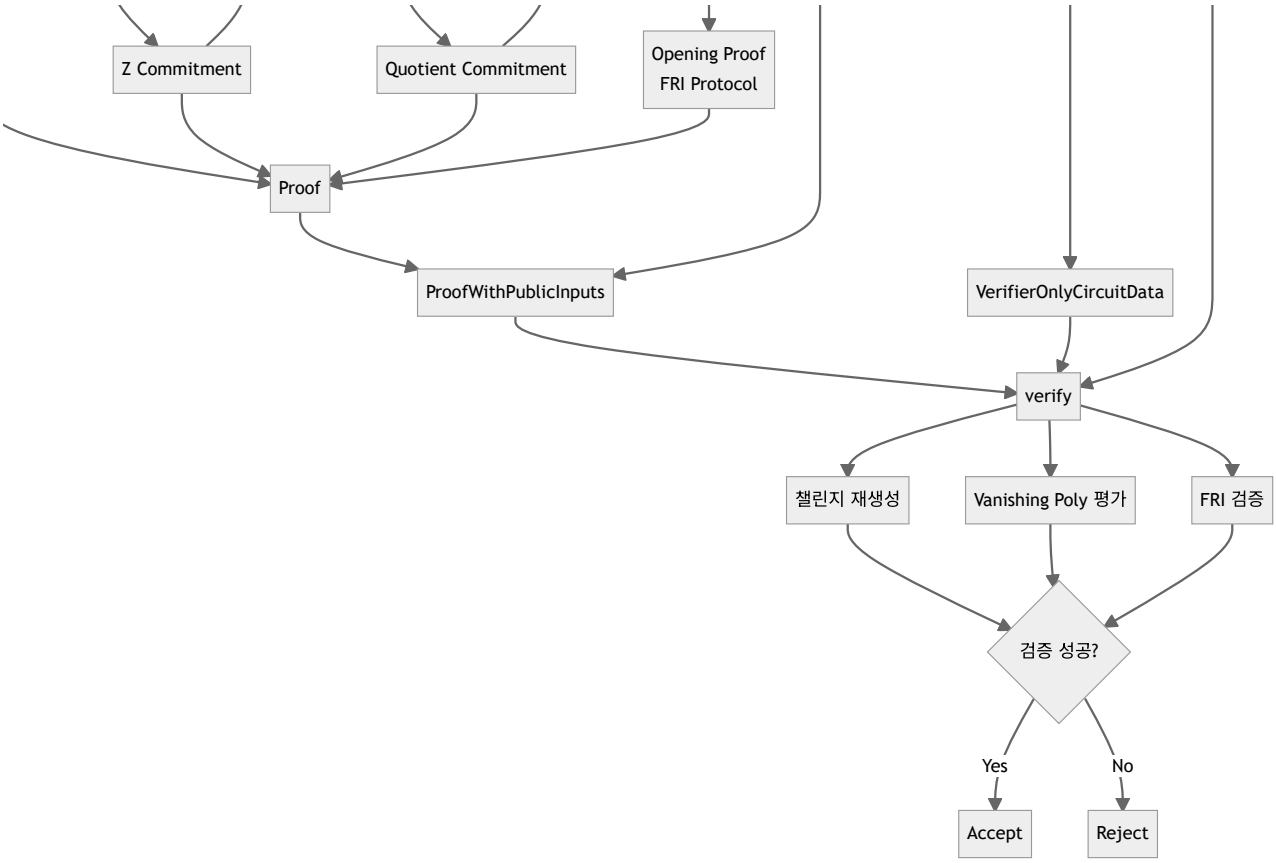
```
+public_inputs: Vec~F~

}


class Proof {

+wires_cap: MerkleCap

+plonk_zs_partial_products_cap: MerkleCap

+quotient_polys_cap: MerkleCap

+openings: OpeningSet

+opening_proof: FriProof

}


class PartialWitness {

+target_values: HashMap~Target, F~

+set_target()

}


CircuitBuilder --> CircuitData : builds

CircuitData --> ProverOnlyCircuitData : contains

CircuitData --> VerifierOnlyCircuitData : contains

CircuitData --> CommonCircuitData : contains

CircuitData --> ProofWithPublicInputs : generates

ProofWithPublicInputs --> Proof : contains
```

```
PartialWitness --> CircuitData : input to prove()
```

전체 데이터 흐름

영지식증명의 생성과 검증 과정을 데이터 흐름 관점에서 설명합니다.


CircuitConfig
CircuitBuilder
add targets, gates, constraints
build
CircuitData
ProverOnlyCircuitData
PartialWitness
CommonCircuitData
prove
generate_partial_witness
PartitionWitness
모든 Wire 값
Wire Commitment
PolynomialBatch + FFT + Merkle Tree
Fiat-Shamir
Challenger
public inputs
Beta, Gamma 챌린지
Alpha 챌린지
Permutation Argument
Z 다항식 계산
Quotient Polynomial 계산
Zeta 챌린지



데이터 흐름 설명

1. **회로 구성 단계**

- `CircuitConfig` 로 `CircuitBuilder` 생성
- Target, Gate, Constraint 추가

- `build()` 호출하여 `CircuitData` 생성

2. **증명 생성 단계**

- `PartialWitness` 에 입력값 설정
- `prove()` 함수 호출
- Witness 생성: `generate_partial_witness()` → `PartitionWitness`
- Wire 값 커밋: FFT + Merkle Tree → `wires_cap`
- Fiat-Shamir: `Challenger` 로 챌린지 생성
- Permutation: Z 다항식 계산 및 커밋 → `plonk_zs_partial_products_cap`
- Quotient: Quotient polynomial 계산 및 커밋 → `quotient_polys_cap`

- Opening: FRI 프로토콜로 opening proof 생성
- 모든 데이터를 `Proof` 로 패키징

3. **증명 검증 단계**

- `ProofWithPublicInputs` 수신
- 챌린지 재생성 (Fiat-Shamir)
- Vanishing polynomial 평가
- FRI 증명 검증
- 모든 검증 통과 시 Accept

주요 프로토콜 흐름

PLONK 프로토콜 흐름

1. **Setup Phase** (회로 빌드 시)

- Gate 선택 및 배치
- Permutation 구조 계산 (σ)
- 상수 및 σ 커밋먼트 생성

2. **Proving Phase**

- Round 1: Wire 값 커밋
- Round 2: Permutation Z 다항식 커밋
- Round 3: Quotient polynomial 커밋
- Round 4: Opening proof (FRI)

3. **Verification Phase**

- 챌린지 재생성

- Polynomial identity 검증
- FRI 검증

FRI 프로토콜 흐름

1. **Commit Phase**

- 다항식 Folding (여러 라운드)
- 각 라운드마다 Merkle Tree 커밋

2. **Query Phase**

- 랜덤 쿼리 포인트 선택
- Merkle Proof 생성

3. **Verification Phase**

- Evaluation 재계산
- Merkle Proof 검증
- Final polynomial 검증

성능 최적화 요소

1. **Goldilocks Field**

- 64비트 연산 최적화
- 높은 2-adicity (FFT 효율성)

2. **병렬 처리**

- Rayon을 통한 데이터 병렬성
- FFT, Merkle Tree 구성 병렬화

3. **메모리 효율성**

- Zero-copy 기법
- Polynomial 재사용

4. **하드웨어 최적화**

- x86-64 SIMD 명령어
- Cache-friendly FFT

다음 단계

이제 plonky2의 전체 아키텍처를 이해했으니, 다음 문서들에서 각 컴포넌트를 상세히 살펴보겠습니다:

- 02-circuit-building.md: CircuitBuilder를 사용한 회로 구성
- 03-proof-generation.md: 증명 생성 프로세스
- 04-proof-verification.md: 증명 검증 프로세스
- 05-fri-protocol.md: FRI 프로토콜 상세
- 06-goldilocks-field.md: Goldilocks Field 구현

참고 자료

- Plonky2 공식 문서
- PLONK 논문
- FRI 논문

문서 네비게이션

- **이전**: 00. 인덱스 - 문서 시리즈 개요

- **다음**: 02. 회로 구성 프로세스 - CircuitBuilder 사용법
- **관련 문서**:
- 06. Goldilocks Field - 기본 필드 구현
- 11. 성능 최적화 - 아키텍처 최적화 기법